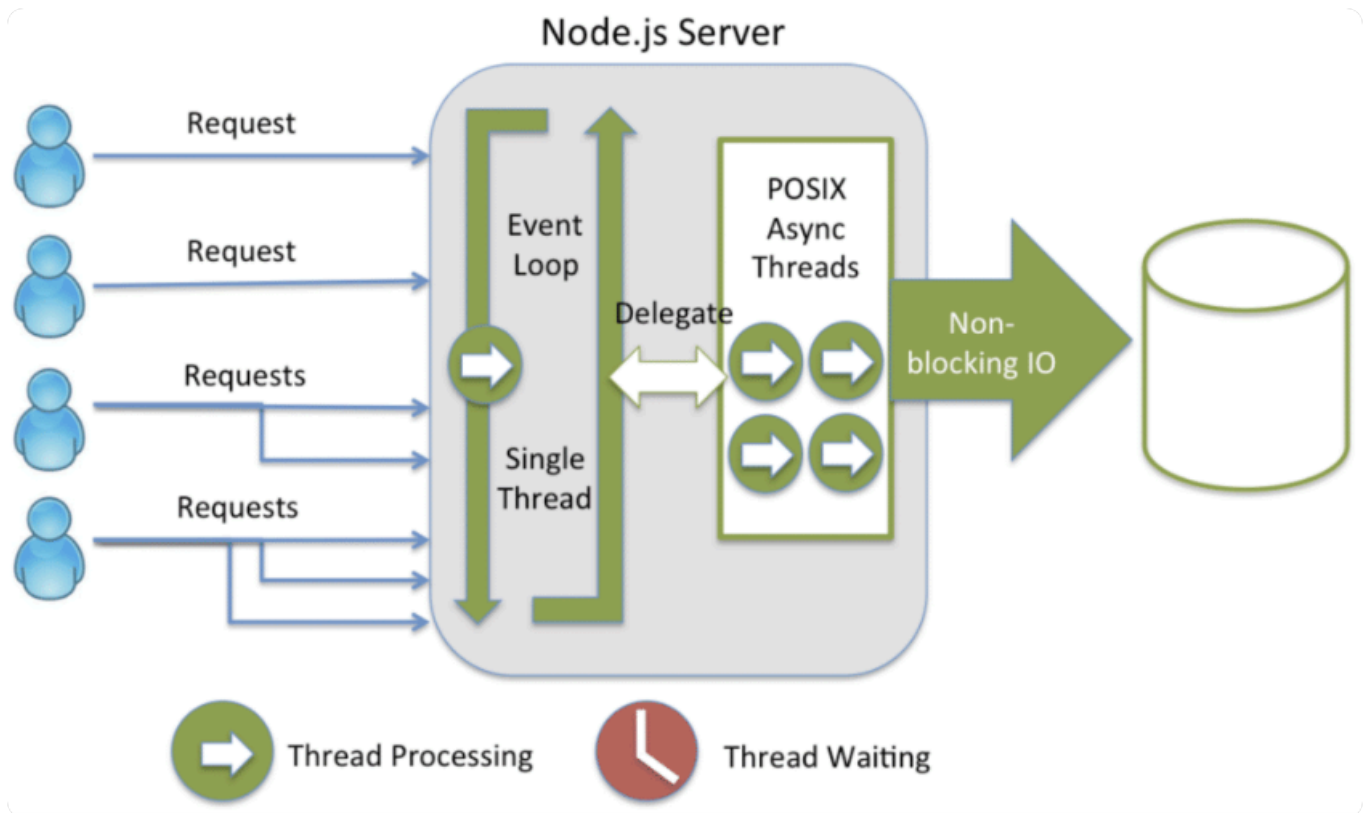


Node.js®是一个基于 Chrome V8 JavaScript 引擎构建的 JavaScript 运行时环境。简单来说，Node.js 让 JavaScript 可以在服务器端运行，而不仅仅局限于浏览器中。



Nodejs发送请求时，进入时间循环，复用统一线程

1.Node.js安装配置（选择LTS长期版本）

Node.js安装包及下载地址：<https://nodejs.org/en/download>

Download Node.js®

Get Node.js® v24.14.1 (LTS) for Windows using Docker with npm

Info Want new features sooner? Get the [latest Node.js version](#) instead and try the latest improvements!

```
1 # Docker has specific installation instructions for each operating system.
2 # Please refer to the official documentation at https://docker.com/get-started/
3
4 # Pull the Node.js Docker image:
5 docker pull node:24-slim
6
7 # Create a Node.js container and start a Shell session:
8 docker run -it --rm --entrypoint sh node:24-slim
9
10 # Verify the Node.js version:
11 node -v # Should print "v24.14.1".
12
13 # Verify npm version:
14 npm -v # Should print "11.11.0".
```

PowerShell

[Copy to clipboard](#)

Docker is a containerization platform. If you encounter any issues please visit [Docker's website](#)

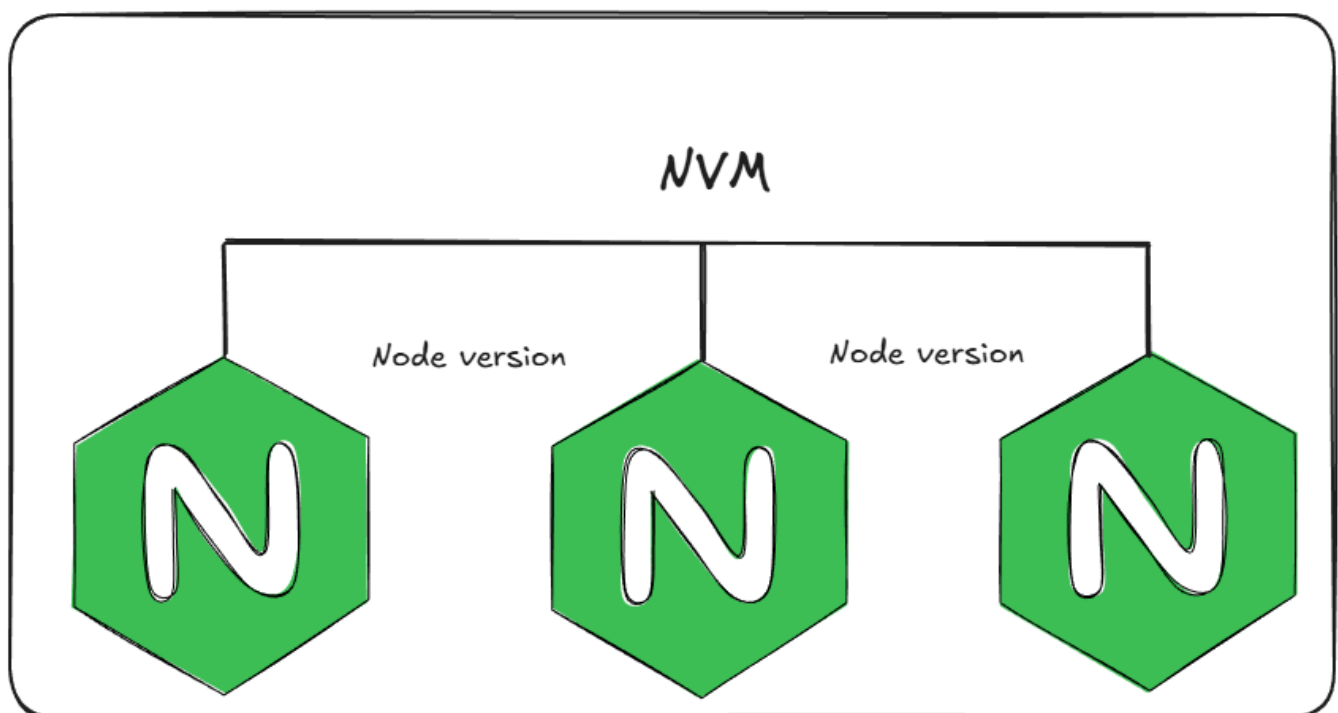
Node.js 历史版本下载地址: <https://nodejs.org/dist/>

```
# 查看node.js安装成功  
node -v
```

2.NVM

NVM是node.js的多版本管理工具, 保证项目对多版本的兼容性

```
# 检查NVM安装成功  
nvm -v  
  
# 查看已有的node.js版本  
nvm list/nvm ls  
  
# 安装最新的LTS版本  
nvm install -lts  
  
# 切换特定版本  
nvm use 18.16.0  
  
# 查看当前使用的版本  
nvm current  
  
# 删除特定版本的node  
nvm uninstall 16.20.1
```



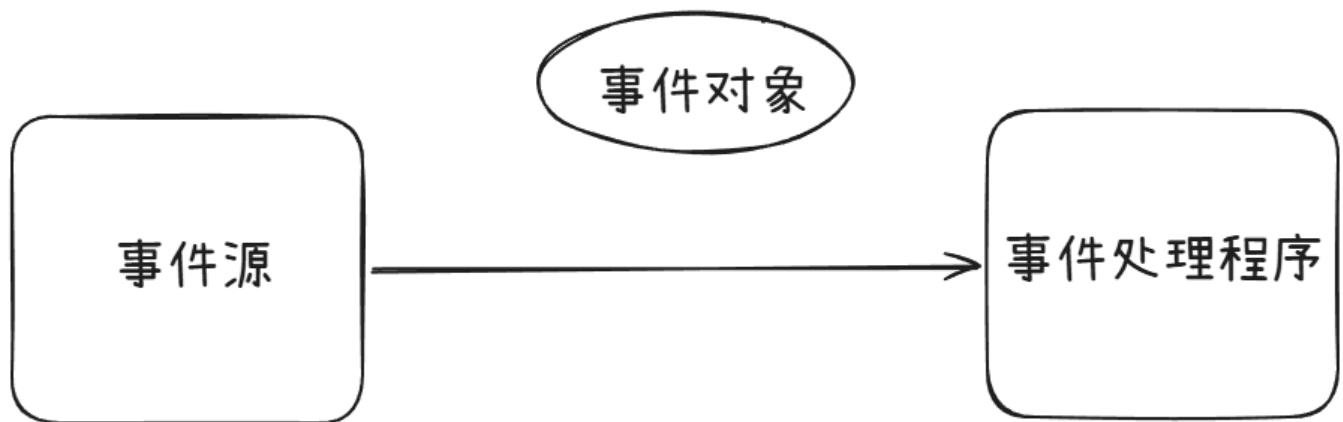
3.Node.js工作原理

- ▶ 高效的事件循环调度
- ▶ 非阻塞I/O操作
- ▶ 单线程但支持多进程/多线程扩展

1.高效的事件循环调度

简单的👤：监听鼠标点击事件，显示鼠标点击的位置坐标

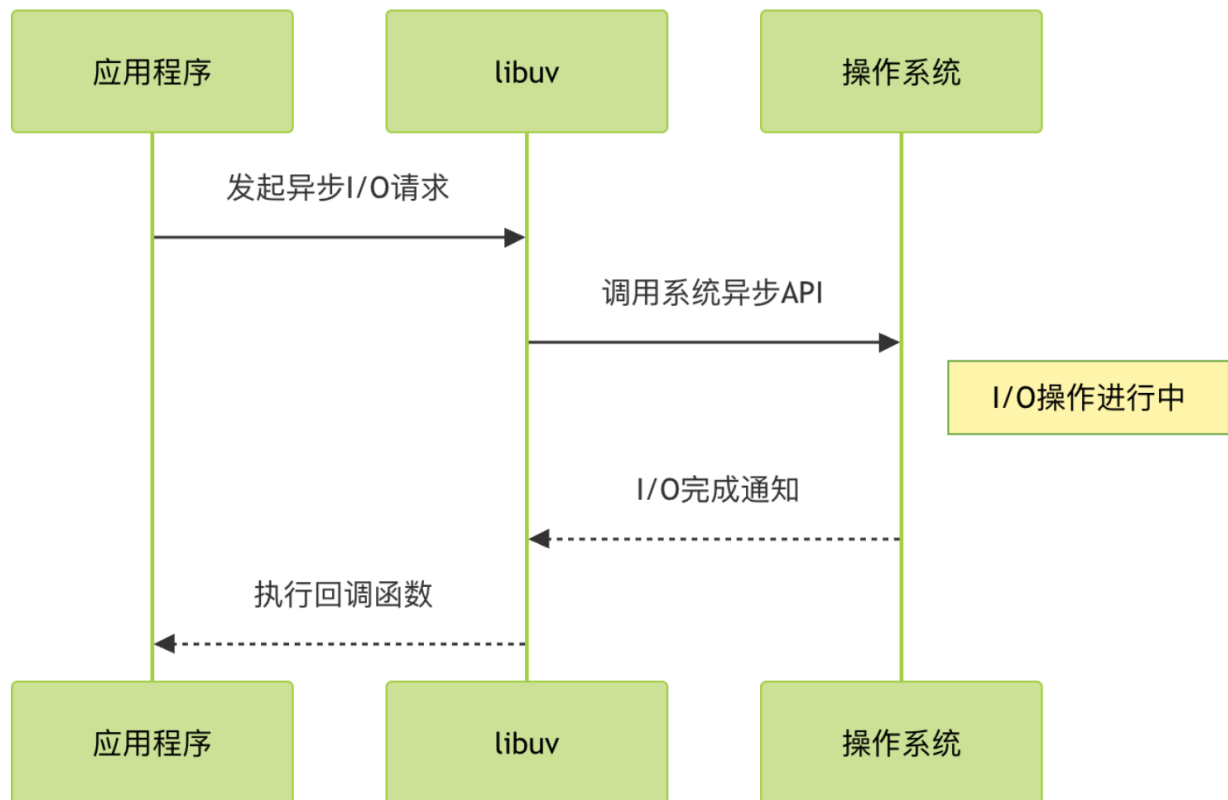
为需要处理的事件编写相应的事件处理程序。要理解事件驱动和程序，就需要与非事件驱动的程序进行比较。实际上，现代的程序大多是事件驱动的，比如多线程的程序，肯定是事件驱动的。早期则存在许多非事件驱动的程序，这样的程序，在需要等待某个条件触发时，会不断地检查这个条件，直到条件满足，这是很浪费cpu时间的。而事件驱动的程序，则有机会释放cpu从而进入睡眠态（注意是有机会，当然程序也可自行决定不释放cpu），当事件触发时被操作系统唤醒，这样就能更加有效地使用cpu。



2.非阻塞I/O操作

- (1) 应用发起 I/O 请求（如读取文件）
- (2) Node.js 将请求交给 libuv 处理
- (3) libuv 使用系统提供的异步接口（如 Linux 的 epoll）
- (4) 主线程继续执行其他任务
- (5) I/O 完成后，回调函数被放入事件队列

- (6) 事件循环在适当阶段执行回调



3.单线程但支持多进程/多线程扩展

Node.js可以通过子进程，集群模式，工作线程来利用多核CPU

Node.js 底层使用了多线程（如 libuv 的线程池处理某些 I/O 操作）

Node.js使用 Worker Threads 或将任务拆分为多个小任务，通过 setImmediate 分批处理

Node.js 适合 I/O 密集型应用,因为它的非阻塞 I/O 模型可以在等待 I/O 时处理其他请求，而不需要为每个连接创建新线程。

4.NPM

NPM（Node Package Manager）是一个 JavaScript 包管理工具，也是 Node.js 的默认包管理器。

NPM 允许开发者轻松地下载、安装、共享、管理项目的依赖库和工具

NPM 是 Node.js 自带的包管理工具，通常你只需安装 Node.js，NPM 就会自动安装在系统中

主要功能：

- **包管理：** NPM 可以帮助你安装并管理项目所需的各种第三方库（包）。例如，可以通过简单的命令来安装、更新、或删除依赖

- **版本管理：**NPM 支持版本控制，允许你锁定某个特定版本的依赖，或根据需求选择最新的版本
- **包发布：**NPM 允许开发者将自己的库发布到 NPM 仓库中，其他开发者可以通过 NPM 下载并使用这些库
- **命令行工具：**NPM 提供了强大的命令行工具，可以用于安装包、运行脚本、初始化项目等多种操作

npm包的安装分为本地安装和全局安装两种，本地安装适用于项目的依赖，全局安装参数是-g，对全局有效作用于系统变量级别，安装的依赖在 node_modules 目录

```
# 本地安装
npm install express

# 全局安装
npm install express -g

# 查看全局安装的模块
npm list -g

# 查看某个模块的版本号
npm list express

# 卸载模块
npm uninstall express

# 更新模块
npm update express

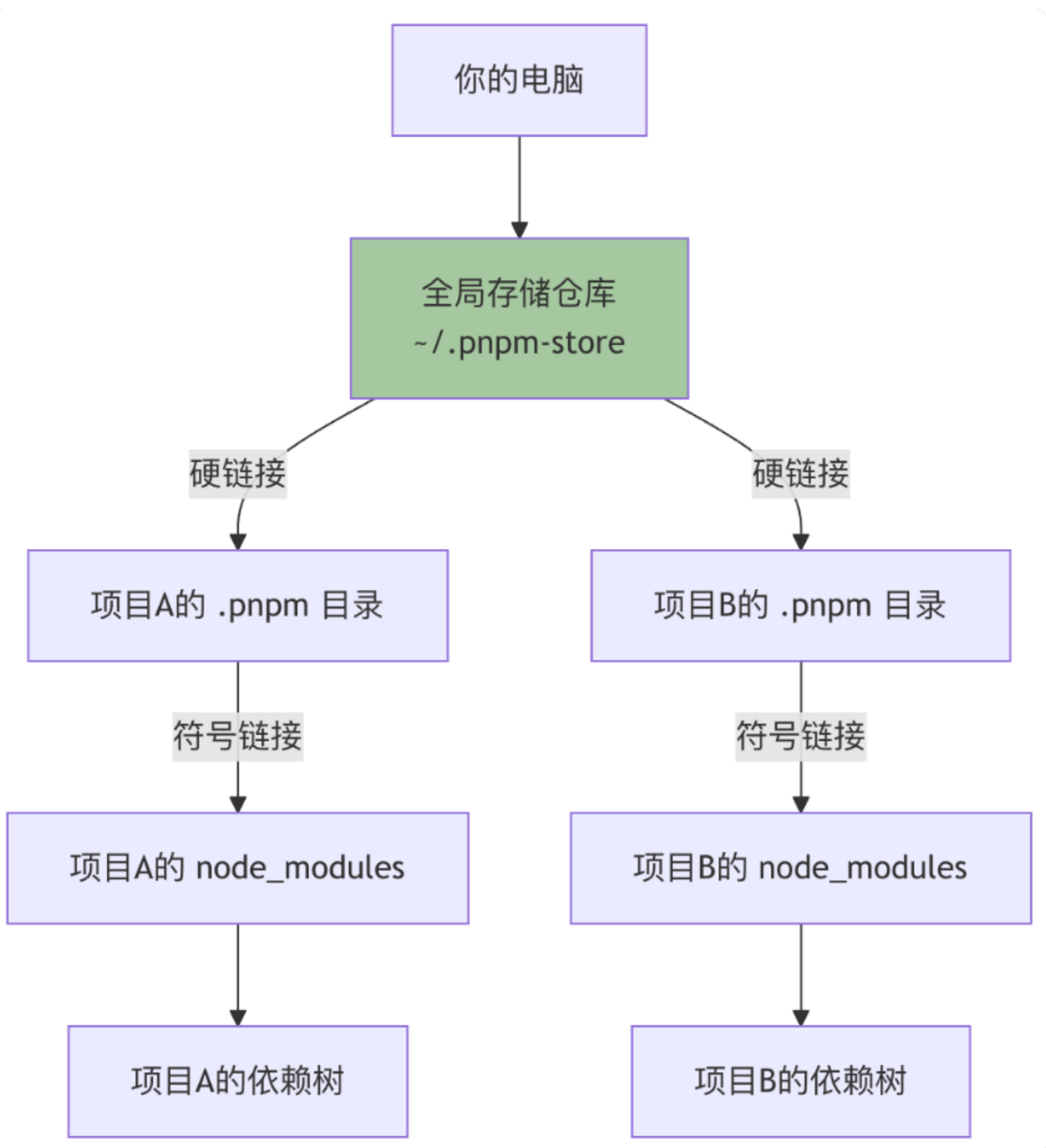
# 搜索模块
npm search express

# 初始化
npm init

# 查看命令的帮助
npm help
```

5.PNPM

pnpm 是一个快速、节省磁盘空间的 Node.js 包管理器，它通过内容寻址的方式存储包，所有版本的依赖会集中存储在系统的一个位置，然后通过硬链接的方式链接到项目中，避免了重复安装



```
# 安装pnpm
npm install -g pnpm

# 查看pnpm安装成功
pnpm -v

# 交互式地回答问题，创建 package.json
pnpm init

# 快速创建一个带有默认值的 package.json
```

```
pnpm init -y
```

安装所有依赖

```
pnpm install
```

添加依赖包

```
pnpm add <package-name>
```

添加开发依赖

```
pnpm add -D <package-name>
```

添加全局包

```
pnpm add -g <package-name>
```

安装指定版本

```
pnpm add <package-name>@<version>
```

更新所有依赖

```
pnpm update
```

更新指定包

```
pnpm update <package-name>
```

更新到最新版本(忽略 package.json 中的版本范围)

```
pnpm update --latest
```

删除依赖

```
pnpm remove <package-name>
```